

LAPORAN TUGAS BESAR
IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 4



Disusun oleh:

Gabriella Botimada Lubis - 13524006

Stefani Angeline Oroh - 13524064

Reva Natania Sitohang - 13524098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR TABEL.....	4
DAFTAR GAMBAR.....	5
BAB I	
DASAR TEORI.....	6
1.1 Arsitektur Compiler dan Interpreter.....	6
1.2 Decorated Abstract Syntax Tree (Decorated AST).....	6
1.3 Intermediate Code Generation.....	7
1.3.1 Konsep Intermediate Code.....	7
1.3.2 Three-Address Code (TAC).....	7
1.3.3 Format Instruksi Stack Machine.....	8
1.3.4 Stack Frame Layout.....	9
1.3.5 Transisi Control Flow dan Backpatching.....	9
1.4 Arsitektur Mesin Eksekusi: Interpreter dan Stack Machine.....	11
1.4.1 Virtual Machine dan Siklus Eksekusi.....	11
1.4.2 Stack sebagai Memori Eksekusi.....	11
1.4.3 Operasi pada Stack Machine.....	12
1.5 Algoritma DFS-based Code Generation.....	13
1.6 Hubungan Antar Komponen dalam Pipeline Atrion.....	13
BAB II	
PERANCANGAN DAN IMPLEMENTASI.....	14
2.1 Arsitektur Program.....	14
2.1.1 Alur Program Secara Keseluruhan.....	14
2.1.2 Arsitektur Back-End Code-Generator.....	15
2.1.3 Arsitektur Back-End Interpreter.....	15
2.1.4 Format Instruksi dan Instruction Set.....	16
2.1.5 Model Activation Record.....	17
2.2 Implementasi Kode.....	17
2.2.1 OpCode dan Struct Instruction.....	17
2.2.2 Fungsi emit() dan patch().....	18
2.2.3 Entry Point Code Generation.....	19
2.2.4 Generasi Kode IF-THEN-ELSE.....	20
2.2.5 Generasi Loop.....	21
2.2.6 Generasi Repeat-Until.....	22
2.2.7 Pemanggilan Prosedur Built-in dan User-Defined.....	24
2.2.8 Generasi Kode untuk Nilai Literal.....	25
2.2.9 Deklarasi Subprogram.....	26
2.2.10 Mesin Eksekusi Utama.....	27

2.2.11 Hierarki Exception dan Safety Checks.....	32
BAB III	
PENGUJIAN.....	34
3.1 Test Case 1.....	34
3.2 Test Case 2.....	36
3.3 Test Case 3.....	38
3.4 Test Case 4.....	40
3.5 Test Case 5.....	42
3.6 Test Case 6.....	43
3.7 Test Case 7.....	45
3.8 Test Case 8.....	46
3.9 Test Case 9.....	48
3.10 Test Case 10.....	51
3.11 Test Case 11.....	52
3.12 Test Case 12.....	54
3.13 Test Case 13.....	55
3.14 Test Case 14.....	57
3.15 Test Case 15.....	59
3.16 Test Case 16.....	62
3.17 Test Case 17.....	63
3.18 Test Case 18.....	64
3.19 Test Case 19.....	65
3.20 Test Case 20.....	66
3.21 Test Case 21.....	67
BAB IV	
KESIMPULAN DAN SARAN.....	68
4.1 Kesimpulan.....	68
4.2 Saran.....	69
LAMPIRAN.....	70
REFERENSI.....	71

DAFTAR TABEL

Tabel 1.3.3 Sembilan Instruksi Stack Machine.....	8
Tabel 1.3.4 Tata Letak Frame.....	9
Tabel 1.4.3 Operasi Stack Machine.....	12
Tabel 2.1.4 (1) Empat Field Struct Instruction.....	16
Tabel 2.1.4 (2) Sembilan OpCode.....	16
Tabel 3.1 Input/Output Test Case 1.....	36
Tabel 3.2 Input/Output Test Case 2.....	38
Tabel 3.3 Input/Output Test Case 3.....	40
Tabel 3.4 Input/Output Test Case 4.....	42
Tabel 3.5 Input/Output Test Case 5.....	43
Tabel 3.6 Input/Output Test Case 6.....	45
Tabel 3.7 Input/Output Test Case 7.....	46
Tabel 3.8 Input/Output Test Case 8.....	48
Tabel 3.9 Input/Output Test Case 9.....	51
Tabel 3.10 Input/Output Test Case 10.....	52
Tabel 3.11 Input/Output Test Case 11.....	54
Tabel 3.12 Input/Output Test Case 12.....	55
Tabel 3.13 Input/Output Test Case 13.....	57
Tabel 3.14 Input/Output Test Case 14.....	59
Tabel 3.13 Input/Output Test Case 15.....	62
Tabel 3.16 Input/Output Test Case 16.....	63
Tabel 3.17 Input/Output Test Case 17.....	64
Tabel 3.18 Input/Output Test Case 18.....	65
Tabel 3.19 Input/Output Test Case 19.....	66
Tabel 3.20 Input/Output Test Case 20.....	67
Tabel 3.21 Input/Output Test Case 21.....	67

DAFTAR GAMBAR

Gambar 2.2.1 Kode OpCode dan Struct Instruction.....	18
Gambar 2.2.2 Kode Fungsi emit() dan patch().....	18
Gambar 2.2.3 Kode genProgram().....	19
Gambar 2.2.4 Kode genIfStmt().....	20
Gambar 2.2.5 Kode genWhileStmt() dan genForStmt().....	21
Gambar 2.2.6 Kode genRepeatStmt() dan genCaseStmt().....	23
Gambar 2.2.7 Kode genProcCall().....	24
Gambar 2.2.8 Kode genLiteral().....	25
Gambar 2.2.9 Kode genProcDecl() dan genFuncDecl().....	26
Gambar 2.2.10 (1) Kode execute().....	28
Gambar 2.2.10 (2) Kode execute().....	30
Gambar 2.2.10 (3) Kode execute().....	31
Gambar 2.2.11 (1) Kode interpreter.hpp.....	32
Gambar 2.2.11 (2) Kode checkArithOverflow(), checkStackSmashing(), checkVariableAccess().....	32

BAB I

DASAR TEORI

1.1 Arsitektur Compiler dan Interpreter

Dalam perancangan sebuah sistem pemroses bahasa pemrograman, arsitektur umumnya dibagi menjadi dua fase besar yang saling terpisah tanggung jawabnya, fase Front-end dan fase Back-end.

Front-end bertugas berinteraksi langsung dengan source code yang ditulis oleh programmer. Ia memotong teks mentah menjadi token (Lexical Analysis), menyusun token menjadi struktur hierarkis, dan memvalidasi kebenaran logika serta tipe data program). Hasil akhir dari seluruh Front-end adalah sebuah struktur data yang sudah bebas dari teks mentah, yaitu Decorated Abstract Syntax Tree (Decorated AST).

Back-end menerima Decorated AST tersebut dan sama sekali tidak peduli dengan sintaksis bahasa asalnya. Back-end berfokus sepenuhnya pada dua tugas, yaitu sintesis yang mengonversi Decorated AST menjadi instruksi operasional yang siap dijalankan, dan eksekusi yang berupa proses mengelola memori dan menjalankan instruksi tersebut pada sebuah mesin virtual.

1.2 Decorated Abstract Syntax Tree (Decorated AST)

Abstract Syntax Tree (AST) biasa hanya merepresentasikan struktur sintaksis dari sebuah program. Setiap node dalam pohon merepresentasikan satu konstruksi bahasa seperti pernyataan assignment, ekspresi aritmatika, atau blok kondisional.

Decorated AST adalah AST yang telah diperkaya dengan informasi semantik pada setiap node-nya. Dua jenis informasi utama yang ditambahkan adalah:

1. **Tipe Data:** Setiap node ekspresi memiliki anotasi tipe data hasil evaluasinya, misalnya Integer, Boolean, atau Void. Informasi ini krusial bagi Code Generator untuk memilih instruksi yang tepat. Tanpa anotasi tipe, generator tidak bisa membedakan apakah ekspresi $a + b$ adalah penjumlahan numerik atau penggabungan string.
2. **Referensi Tabel Simbol:** Setiap node yang merepresentasikan sebuah variabel memiliki referensi langsung ke entri variabel tersebut di dalam Symbol Table. Referensi ini mengandung informasi penting seperti lexical level dan address offset yang keduanya dibutuhkan untuk membangkitkan instruksi LOD dan STO yang benar

Tabel Simbol pada implementasi Arion Interpreter terdiri dari tiga komponen:

1. TAB (Identifier Table)

Menyimpan setiap entri identifier (variabel, konstanta, fungsi, prosedur) beserta atributnya seperti nama, jenis objek, tipe data, lexical level (lev), dan address offset (adr).

2. BTAB (Block Table)

Menyimpan informasi per blok scope, termasuk psze (jumlah parameter) dan vsze (jumlah variabel lokal). Informasi ini digunakan untuk menghitung ukuran frame memori yang tepat.

3. ATAB (Array Table)

Menyimpan informasi dimensi dan batas indeks array jika terdapat deklarasi tipe array di dalam program.

1.3 Intermediate Code Generation

1.3.1 Konsep Intermediate Code

Intermediate Code (kode perantara) adalah representasi program dalam bentuk instruksi linear dan sederhana yang menjembatani antara Decorated AST (representasi konseptual bertingkat) dengan eksekusi nyata oleh mesin. Kode ini disebut perantara karena tidak setinggi bahasa sumber dan tidak serendah kode mesin yang sesungguhnya.

Keberadaan Intermediate Code memberikan beberapa manfaat. Pertama, instruksi linear jauh lebih efisien dieksekusi dibanding harus melompat-lompat di dalam struktur pohon. Kedua, Intermediate Code yang sama dapat dieksekusi di berbagai platform yang berbeda selama tersedia interpreter untuk platform tersebut. Ketiga, Intermediate Code dapat menjadi target optimasi sebelum eksekusi akhir.

1.3.2 Three-Address Code (TAC)

Format Intermediate Code yang paling luas digunakan adalah Three-Address Code (TAC). Dinamakan demikian karena setiap instruksi dirancang seminimal mungkin: paling banyak hanya melibatkan tiga alamat memori, yang biasanya terdiri dari dua operand sumber dan satu lokasi tujuan. Struktur dasar TAC adalah:

```
hasil := operand1 operator operand2
```

Ketika ekspresi sumber sangat kompleks dan bersarang, TAC memecahnya menjadi rangkaian operasi biner sederhana dengan memanfaatkan variabel sementara

(biasanya dinotasikan t_1 , t_2 , t_3 , dst.). Sebagai contoh, ekspresi $x := (a + b) * (c - d)$ dalam Arion ditranslasikan menjadi:

$t_1 := a + b$	{ Langkah 1: Evaluasi kurung pertama }
$t_2 := c - d$	{ Langkah 2: Evaluasi kurung kedua }
$t_3 := t_1 * t_2$	{ Langkah 3: Kalikan hasil keduanya }
$x := t_3$	{ Langkah 4: Simpan hasil akhir }

Proses perataan (flattening) ini dilakukan dengan cara menelusuri AST dari cabang terdalam ke akar, sehingga setiap subekspresi dievaluasi dan hasilnya disimpan sebelum digunakan oleh ekspresi induknya. Pendekatan ini merupakan penerapan prinsip Depth-First Search (DFS) pada struktur pohon.

1.3.3 Format Instruksi Stack Machine

Pada implementasi Arion Interpreter, TAC direpresentasikan dalam format Stack Machine Instructions yang lebih konkret dan langsung dapat dieksekusi oleh interpreter. Setiap instruksi memiliki format:

[nomor_baris] [INSTRUKSI] [level] [operand]

Sembilan instruksi yang didukung ditunjukkan pada tabel berikut:

No	Instruksi	Operasi	Keterangan
1	LIT v	Load Literal	Push nilai literal v ke stack evaluasi
2	LOD a	Load Value	Push nilai variabel di alamat a dari frame aktif
3	STO a	Store Value	Pop nilai dari stack, simpan ke alamat a
4	CAL l	Call	Panggil prosedur/fungsi di baris instruksi l
5	INT m	Initiate Memory	Alokasi frame memori baru sebesar m slot
6	JMP l	Unconditional Jump	Set instruction pointer ke baris l tanpa syarat
7	JPC l	Conditional Jump	Set IP ke l jika nilai teratas stack = 0 (false)
8	OPR o	Operation	Jalankan operasi nomor o pada nilai-nilai di stack
9	RET	Return	Keluar dari prosedur/fungsi, kembali ke pemanggil

Tabel 1.3.3 Sembilan Instruksi Stack Machine

1.3.4 Stack Frame Layout

Setiap kali sebuah program atau prosedur dieksekusi, interpreter mengalokasikan sebuah frame memori di stack. Frame ini memiliki tata letak standar sebagai berikut:

Posisi	Isi	Keterangan
Slot 0	Static Link	Pointer ke frame dari scope statis tempat prosedur dideklarasikan
Slot 1	Dynamic Link	Pointer ke frame pemanggil, digunakan saat RET untuk kembali
Slot 2	Return Address	Nomor baris instruksi untuk dilanjutkan setelah RET
Slot 3 s.d. $3+psze-1$	Parameter	Argumen yang diteruskan ke prosedur/fungsi
Slot $3+psze$ s.d. $3+psze+vsze-1$	Local Variables	Variabel yang dideklarasikan di dalam blok

Tabel 1.3.4 Tata Letak Frame

Sehingga ukuran total frame adalah $3 + psze + vsze$, dan instruksi INT 0 m mengalokasikan tepat m slot tersebut. Alamat stack absolut sebuah variabel dihitung sebagai:

- Parameter: $base + 3 + adr$
- Variabel lokal: $base + 3 + psze + adr$

di mana $base$ adalah posisi awal frame aktif di stack, dan adr adalah offset relatif dari TAB (Identifier Table).

1.3.5 Transisi Control Flow dan Backpatching

Salah satu tantangan terbesar dalam Intermediate Code Generation adalah meratakan struktur kontrol bersarang (IF-ELSE, WHILE, FOR, REPEAT-UNTIL) menjadi instruksi linear. Solusinya adalah kombinasi label dan instruksi jump. Pada implementasi stack machine, label tidak ditulis secara eksplisit melainkan direpresentasikan sebagai nomor baris instruksi.

Instruksi JPC digunakan sebagai conditional jump , dan JMP sebagai unconditional jump. Konsep JPC (jump if false) dipilih berdasarkan prinsip fall-through: ketika kondisi bernilai benar, eksekusi dapat langsung berlanjut ke instruksi berikutnya tanpa perlu lompatan tambahan. Lompatan hanya terjadi saat kondisi bernilai salah, sehingga alur kontrol menjadi lebih sederhana.

Pola translasi untuk konstruksi kontrol utama:

- IF-ELSE

```
[generate kondisi]
JPC 0 awal_else      ; lompat ke else jika
kondisi false
[generate then-block]
JMP 0 setelah_else   ; lewati else setelah then
selesai
[generate else-block] ; awal_else
                        ; setelah_else
```

- WHILE

```
loop_start:
[generate kondisi]
JPC 0 setelah_loop   ; keluar loop jika kondisi
false
[generate body]
JMP 0 loop_start      ; kembali ke evaluasi
kondisi
                        ; <- setelah_loop
```

- REPEAT-UNTIL

```
loop_start:
[generate body]
[generate kondisi UNTIL]
JPC 0 loop_start      ; kembali jika kondisi
UNTIL masih false
```

Karena nomor baris instruksi target seringkali belum diketahui saat instruksi JPC atau JMP pertama kali di-emit, digunakan teknik backpatching: instruksi jump di-emit dengan target sementara (0), kemudian setelah target yang benar diketahui, operand instruksi tersebut diperbarui. Dalam implementasi, ini dilakukan oleh fungsi emit() yang

mengembalikan index instruksi, dan fungsi patch(index, target) yang mengisi target yang tepat.

1.4 Arsitektur Mesin Eksekusi: Interpreter dan Stack Machine

1.4.1 Virtual Machine dan Siklus Eksekusi

Interpreter dalam konteks ini berperan sebagai sebuah Virtual Machine (VM): sebuah mesin eksekusi perangkat lunak yang tidak memiliki wujud fisik namun memiliki logika memori dan siklus kerja yang setara dengan CPU sungguhan.

Komponen utama VM adalah Instruction Pointer (IP) atau Program Counter (PC), yaitu sebuah penunjuk yang melacak baris instruksi mana yang sedang atau akan dieksekusi. IP bergerak maju satu langkah setiap siklus, kecuali saat instruksi JMP, JPC, atau CAL mengubahnya secara eksplisit.

VM beroperasi dalam siklus eksekusi tiga tahap yang berulang terus-menerus hingga instruksi RET terakhir dijumpai:

1. **Fetch:** Membaca instruksi yang ditunjuk oleh IP dari daftar instruksi TAC.
2. **Decode:** Mengidentifikasi jenis operasi (LIT, LOD, STO, OPR, dsb.) dan mengekstrak nilai operannya.
3. **Execute:** Melakukan aksi yang sesuai. Menghitung nilai, memperbarui memori, mencetak output, atau mengubah IP untuk implementasi lompatan.

1.4.2 Stack sebagai Memori Eksekusi

Struktur memori utama yang digunakan oleh VM adalah stack, sebuah struktur data yang bekerja dengan prinsip LIFO (Last In, First Out). Stack digunakan untuk dua tujuan yang saling berkaitan:

Pertama, sebagai evaluation stack tempat nilai-nilai sementara ditampung selama evaluasi ekspresi. Ketika instruksi LOD atau LIT dieksekusi, nilai di-push ke puncak stack. Ketika OPR dieksekusi, satu atau dua nilai di-pop dari stack, dihitung, dan hasilnya di-push kembali.

Kedua, sebagai call stack yang mengelola frame memori dari setiap prosedur atau fungsi yang aktif. Setiap pemanggilan (CAL) membuat frame baru di stack, dan setiap pengembalian (RET) menghancurkan frame tersebut, mengembalikan kendali ke pemanggil secara otomatis.

Stack secara alami mengimplementasikan konsep scope dan lifetime variabel: variabel lokal sebuah prosedur hanya ada selama prosedur tersebut aktif (frame-nya berada di stack), dan otomatis tidak dapat diakses lagi setelah prosedur selesai dan frame-nya di-pop.

1.4.3 Operasi pada Stack Machine

Instruksi OPR merupakan instruksi serbaguna yang mengeksekusi 14 jenis operasi aritmatika, relasional, dan output:

No	Nama	Operasi	Keterangan
1	NEG	Negasi nilai	Membalikkan tanda nilai teratas stack (-x)
2	ADD	Penjumlahan	Menambah dua nilai teratas stack
3	SUB	Pengurangan	Mengurangi dua nilai teratas stack
4	MUL	Perkalian	Mengalikan dua nilai teratas stack
5	DIV	Pembagian	Membagi dua nilai teratas stack (integer)
6	MOD	Modulus	Sisa bagi dua nilai teratas stack
7	EQL	Sama dengan (=)	Push 1 jika sama, 0 jika tidak
8	NEQ	Tidak sama (<>)	Push 1 jika tidak sama, 0 jika sama
9	LSS	Kurang dari (<)	Push 1 jika operand kiri < kanan
10	GEQ	Lebih dari sama (>=)	Push 1 jika operand kiri >= kanan
11	GTR	Lebih dari (>)	Push 1 jika operand kiri > kanan
12	LEQ	Kurang dari sama (<=)	Push 1 jika operand kiri <= kanan
13	WRT	Write	Cetak nilai teratas stack (tanpa newline)
14	WRTLN	Writeln	Cetak nilai teratas stack (dengan newline)

Tabel 1.4.3 Operasi Stack Machine

Semua operasi biner (ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ) mengikuti pola yang sama: dua nilai di-pop dari stack, operasi diterapkan dengan operand kiri sebagai nilai yang lebih dulu di-push dan operand kanan sebagai nilai yang terakhir di-push, kemudian hasilnya di-push kembali ke stack.

1.5 Algoritma DFS-based Code Generation

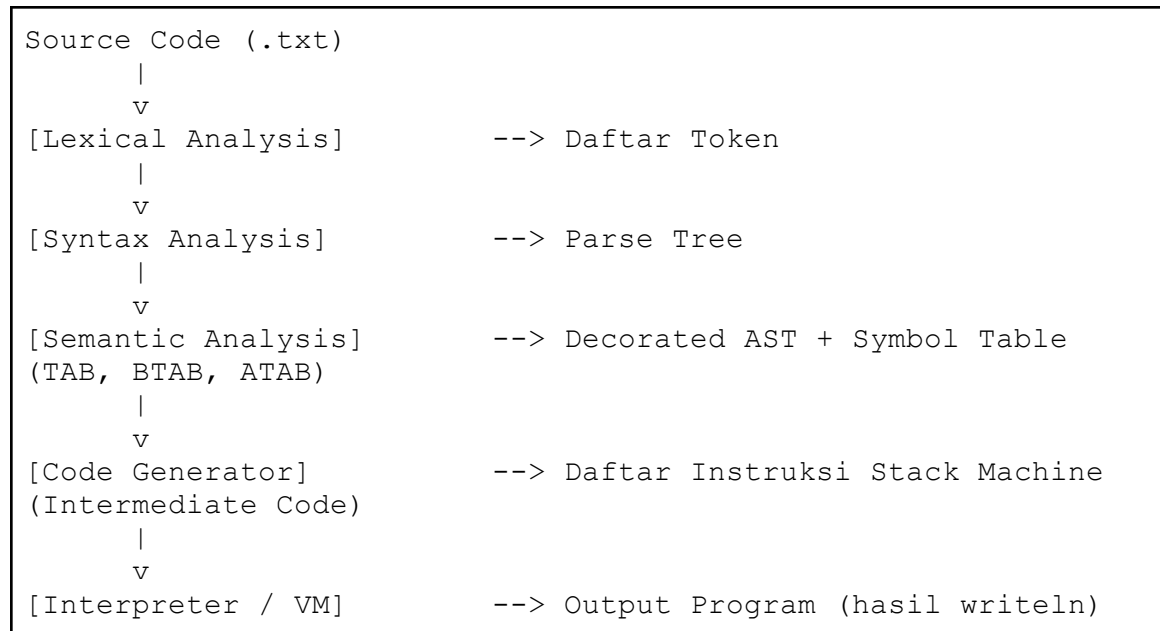
Proses pembangkitan instruksi dari Decorated AST menggunakan pendekatan berbasis Depth-First Search (DFS). Traversal dimulai dari node akar (Program), kemudian secara rekursif menelusuri setiap anak node sebelum kembali ke node induk.

Karakteristik penting dari pendekatan ini adalah urutan evaluasi post-order untuk ekspresi: operand kiri dan kanan sebuah operasi biner di-generate lebih dahulu sebelum instruksi operasinya di-emit. Hal ini memastikan bahwa saat instruksi OPR dieksekusi, kedua operannya sudah tersedia di puncak stack dalam urutan yang benar.

Untuk statement-statement, traversal dilakukan secara in-order sesuai urutan kemunculan di AST, yang merepresentasikan urutan eksekusi program yang diharapkan. Setiap jenis node AST ditangani oleh satu fungsi generator yang terdedikasi (genAssign, genIfStmt, genWhileStmt, dsb.), menjaga agar kode tetap modular dan mudah dipelihara.

1.6 Hubungan Antar Komponen dalam Pipeline Arion

Pipeline lengkap Arion Interpreter menghubungkan seluruh komponen dari Milestone 1 hingga Milestone 4 sebagai berikut:



Setiap tahap menerima output dari tahap sebelumnya dan menghasilkan representasi yang semakin rendah tingkatnya: dari teks mentah, ke token, ke pohon sintaksis, ke pohon semantik, ke instruksi linear, hingga akhirnya ke output nyata di layar. Milestone 4 mengimplementasikan dua tahap terakhir dari pipeline ini, yaitu Intermediate Code Generator dan Interpreter.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1 Arsitektur Program

2.1.1 Alur Program Secara Keseluruhan

Program Arion Interpreter merupakan implementasi lengkap dari sebuah interpreter bahasa pemrograman berbasis Pascal. Pada Milestone 4 ini, program telah dilengkapi dengan dua komponen back-end utama yang bekerja setelah pipeline front-end (Lexer, Parser, Semantic Analyser) selesai, yaitu Intermediate Code Generator dan Interpreter (Executor). Secara keseluruhan, alur pemrosesan program berjalan melalui beberapa fase berurutan sebagai berikut:

1. Pengguna menyediakan file teks berisi kode sumber bahasa Arion (berbasis Pascal).
2. Mengubah karakter-karakter sumber menjadi aliran token. Token mencakup identifier, literal, operator, dan kata kunci.
3. Menyusun token-token tersebut menjadi Parse Tree berdasarkan tata bahasa konteks bebas (CFG) bahasa Arion.
4. Menghias Parse Tree menjadi Decorated AST (Abstract Syntax Tree). Proses ini melakukan type checking, resolusi simbol, dan validasi semantik, menghasilkan Symbol Table (SymbolTable) yang menjadi sumber kebenaran bagi back-end.
5. Men traversasi Decorated AST dan menghasilkan urutan instruksi untuk mesin stack virtual (P-Machine). Output berupa list of Instruction dengan format: nomor baris, OpCode, level, dan operand.
6. Mengeksekusi instruksi-instruksi intermediate code tersebut menggunakan stack virtual. Mengelola stack memori, activation record, dan menghasilkan output program.

Selama front-end menghasilkan Decorated AST dengan format yang sama, back-end dapat mengeksekusinya tanpa perubahan, yang memungkinkan *reuse* mesin eksekusi untuk dialek bahasa lain di masa depan.

2.1.2 Arsitektur Back-End Code-Generator

Code Generator diimplementasikan dalam kelas CodeGenerator dengan nama file codegenerator/codegen.hpp dan codegen.cpp. Komponen ini menerima dua input utama:

1. Decorated AST (ASTPtr): struktur pohon yang telah dianotasi oleh Semantic Analyser, berisi informasi tipe data, tabIndex ke symbol table, dan blockIndex.
2. Symbol Table (SymbolTable): berisi semua entry simbol (variabel, konstanta, prosedur, fungsi, parameter) beserta atribut masing-masing (level, alamat, ukuran frame, dll).

Code Generator menghasilkan dua output:

1. List instruksi (vector<Instruction>): daftar instruksi mesin virtual berformat PL/0-style (OpCode, level, operand).
2. String table (vector<string>): tabel literal string yang digunakan instruksi LIT untuk mereferensikan string secara efisien.

2.1.3 Arsitektur Back-End Interpreter

Interpreter diimplementasikan dalam kelas Interpreter, dengan nama file interpreter/interpreter.hpp dan interpreter.cpp. Komponen ini menerima tiga input, yaitu:

1. List instruksi dari Code Generator.
2. Symbol Table dari Semantic Analyser (untuk keperluan address mapping dan validasi).
3. String Table dari Code Generator.

Interpreter menggunakan model eksekusi Stack Machine dengan komponen memori utama:

- Stack array (StackVal stack_[32768]): stack memori tunggal berkapasitas 32.768 slot untuk menyimpan nilai integer dan referensi string.
- Program Counter (pc_): petunjuk instruksi saat ini.
- Base pointer (base_): penunjuk ke base activation record frame yang sedang aktif.
- Top pointer (top_): penunjuk ke top of stack saat ini.
- Call depth counter (callDepth_): penghitung kedalaman pemanggilan fungsi/prosedur untuk deteksi infinite recursion.

2.1.4 Format Instruksi dan Instruction Set

Setiap instruksi direpresentasikan sebagai struct Instruction dengan 4 field:

Field	Tipe	Contoh	Keterangan
line	int	0, 1, 2, ...	Nomor instruksi (indeks dalam code array)
op	OpCode	LIT, LOD, ...	Kode operasi instruksi
level	int	0, 1, 2	Perbedaan level leksikal (untuk akses variabel outer scope)
operand	int	0, 3, 42	Nilai / alamat / indeks operand

Tabel 2.1.4 (1) Empat Field Struct Instruction

Selain itu, terdapat 9 OpCode yang didukung, antara lain:

OpCode	Fungsi	Detail Operand
LIT	Push literal ke stack	level=0: push integer; level=1: push string (indeks ke stringTable)
LOD	Load variabel ke stack	level=perbedaan scope; operand=alamat relatif dalam frame
STO	Store nilai dari stack ke variabel	level=perbedaan scope; operand=alamat relatif dalam frame
INT	Alokasi frame (activation record)	operand=ukuran frame (3 + psze + vsze)
JMP	Unconditional jump	operand=alamat instruksi tujuan
JPC	Conditional jump jika false (ival==0)	operand=alamat instruksi tujuan
OPR	Operasi aritmetika/logika/I/O	operand 0-14: NEG, ADD, SUB, MUL, DIV, MOD, EQ, NEQ, LT, GEQ, GT, LEQ, WRT, WRITELN
CAL	Panggil prosedur/fungsi	level=perbedaan scope; operand=alamat awal prosedur/fungsi
RET	Return dari prosedur/fungsi	operand=0 (prosedur) atau 1 (fungsi, ada nilai balik)

Tabel 2.1.4 (2) Sembilan OpCode

2.1.5 Model Activation Record

Setiap kali prosedur atau fungsi dipanggil, Interpreter membuat activation record baru di stack. Format frame yang digunakan adalah

```
[base+0]  SL (Static Link)      : base dari outer scope
[base+1]  DL (Dynamic Link)     : base dari caller
[base+2]  RA (Return Address)   : PC setelah instruksi CAL
[base+3]  Parameter 0           : argumen pertama
[base+4]  Parameter 1           : argumen kedua
...                               : dst.
[base+3+psze] Variabel lokal 0
[base+3+psze+1] Variabel lokal 1
...                               : dst.
Ukuran frame = 3 + psze (jumlah parameter) + vsze (jumlah
variabel lokal)
```

2.2 Implementasi Kode

2.2.1 OpCode dan Struct Instruction

```
1  enum class OpCode {
2      LIT,
3      LOD,
4      STO,
5      CAL,
6      INT,
7      JMP,
8      JPC,
9      OPR,
10     RET,
11 };
12
13 std::string opCodeToString(OpCode op);
14
15 struct Instruction {
16     int line;
17     OpCode op;
18     int level;
19     int operand;
20
21     Instruction(int ln, OpCode o, int lv, int oper) : line(ln), op(o), level(lv), operand(oper) {}
22 };
23
24 class CodeGenError : public std::runtime_error {
25 public:
26     explicit CodeGenError(const std::string& msg) : std::runtime_error("[CODEGEN ERROR] " + msg) {}
27 };
```

Gambar 2.2.1 Kode OpCode dan Struct Instruction

Bagian ini mendefinisikan semua instruksi mesin virtual (OpCode) dan format penyimpanannya. enum class OpCode mendefinisikan 9 jenis instruksi mesin virtual, yaitu LIT (push literal), LOD (load variabel), STO (store variabel), CAL (call), INT (alokasi frame), JMP (lompat tanpa syarat), JPC (lompat bersyarat), OPR (operasi), RET (return). Kemudian, struct Instruction menyimpan satu instruksi dengan empat field, yaitu line (nomor instruksi), op (jenis instruksi), level (perbedaan level leksikal), dan operand (nilai/alamat operan). Kelas CodeGenError mewarisi std::runtime_error untuk melaporkan kesalahan selama code generation.

2.2.2 Fungsi emit() dan patch()



```
1 int CodeGenerator::emit(OpCode op, int level, int operand) {
2     int ln = nextLine();
3     code_.emplace_back(ln, op, level, operand);
4     return ln;
5 }
6
7 void CodeGenerator::patch(int instrIndex, int newOperand) {
8     if (instrIndex < 0 || instrIndex >= (int)code_.size())
9         throw CodeGenError("patch: index instruksi di luar batas (" + std::to_string(instrIndex) + ")");
10    code_[instrIndex].operand = newOperand;
11 }
```

Gambar 2.2.2 Kode Fungsi emit() dan patch()

Fungsi emit(OpCode op, int level, int operand) berfungsi untuk menambahkan satu instruksi baru ke akhir list instruksi (code_). Mengembalikan indeks (nomor baris) instruksi yang baru dibuat, yang digunakan di hampir semua fungsi genXxx(). Selanjutnya, patch(int instrIndex, int newOperand) untuk mengubah operand dari instruksi yang sudah ada di posisi instrIndex. Teknik ini penting untuk backpatching, saat instruksi JPC atau JMP dibuat, alamat tujuannya belum diketahui (diisi 0), lalu setelah blok terkait selesai di-generate, patch() dipanggil untuk mengisi alamat yang benar.

2.2.3 Entry Point Code Generation

```
1 void CodeGenerator::genProgram(const ASTPtr& node) {
2     curLev_ = 0;
3     int fsize = frameSize(0);
4     emit(OpCode::INT, 0, fsize);
5     ASTPtr declBlock = nullptr;
6     ASTPtr bodyBlock = nullptr;
7
8     for (const auto& child : node->children) {
9         if (child->kind == ASTNodeKind::BLOCK && child->name == "declarations") {
10             declBlock = child;
11         } else if (child->kind == ASTNodeKind::BLOCK || child->kind == ASTNodeKind::COMPOUND) {
12             bodyBlock = child;
13         }
14     }
15
16     bool hasSubprog = false;
17     if (declBlock) {
18         for (const auto& d : declBlock->children) {
19             if (d->kind == ASTNodeKind::PROC_DECL || d->kind == ASTNodeKind::FUNC_DECL) {
20                 hasSubprog = true;
21                 break;
22             }
23         }
24     }
25
26     // JMP ke body
27     int jmpToBody = -1;
28     if (hasSubprog) {
29         jmpToBody = emit(OpCode::JMP, 0, 0);
30     }
31
32     // generate deklarasi subprogram (proc/func)
33     if (declBlock) {
34         for (const auto& d : declBlock->children) {
35             if (d->kind == ASTNodeKind::PROC_DECL) genProcDecl(d);
36             else if (d->kind == ASTNodeKind::FUNC_DECL) genFuncDecl(d);
37             // VarDecl tidak menghasilkan instruksi (sudah di-handle oleh INT)
38         }
39     }
40
41     // JMP ke posisi body sekarang
42     if (jmpToBody >= 0) {
43         patch(jmpToBody, nextLine());
44     }
45
46     // generate body program utama
47     if (bodyBlock) {
48         genCompound(bodyBlock);
49     }
50     emit(OpCode::RET, 0, 0);
51 }
```

Gambar 2.2.3 Kode genProgram()

Berikut merupakan penjelasan dari alur kode genProgram():

- Memulai dengan emit INT 0, frameSize(0) untuk mengalokasikan *activation record* program utama (level 0).
- Melakukan scan apakah ada subprogram (procedure/function) dalam deklarasi. Jika ada, emit JMP ke body utama terlebih dahulu agar eksekusi tidak langsung jatuh ke kode subprogram.

- Men-generate kode untuk setiap deklarasi subprogram (genProcDecl, genFuncDecl). Deklarasi variabel biasa tidak menghasilkan instruksi karena sudah di-handle oleh INT.
- Setelah semua subprogram ter-generate, patch alamat JMP ke posisi instruksi body program utama.
- Men-generate body program utama melalui genCompound(), kemudian menutup dengan emit RET 0, 0.

2.2.4 Generasi Kode IF-THEN-ELSE

Implementasi generasi kode untuk pernyataan kondisional IF-THEN-ELSE menggunakan teknik backpatching.



```

1 void CodeGenerator::genIfStmt(const ASTPtr& node) {
2     if (node->children.empty())
3         throw CodeGenError("genIfStmt: node IF_STMT tidak memiliki anak.");
4
5     const ASTPtr& cond = node->children[0];
6     ASTPtr thenBlock = (node->children.size() > 1) ? node->children[1] : nullptr;
7     ASTPtr elseBlock = (node->children.size() > 2) ? node->children[2] : nullptr;
8
9     genExpr(cond);
10    int jpcIdx = emit(OpCode::JPC, 0, 0);
11    if (thenBlock) genStatement(thenBlock);
12
13    if (elseBlock) {
14        int jmpIdx = emit(OpCode::JMP, 0, 0);
15        patch(jpcIdx, nextLine());
16        genStatement(elseBlock);
17        patch(jmpIdx, nextLine());
18    } else {
19        patch(jpcIdx, nextLine());
20    }
21 }

```

Gambar 2.2.4 Kode genIfStmt()

Berikut merupakan penjelasan dari kode genIfStmt():

- genExpr(cond) men-generate kode ekspresi kondisi. Hasilnya ditaruh di atas stack.
- emit JPC 0, 0 adalah instruksi conditional jump (JPC = Jump if Condition is false/zero). Alamat tujuan belum diketahui, diisi 0 sementara. Indeksnya disimpan ke jpcIdx.
- genStatement(thenBlock) men-generate kode blok THEN.

- Jika ada ELSE, emit JMP 0, 0 (untuk melewati blok ELSE setelah THEN selesai). Lalu patch(jpcIdx, nextLine()) untuk mengarahkan JPC ke awal ELSE. Setelah generate blok ELSE, patch JMP ke setelah ELSE.
- Jika tanpa ELSE, cukup patch(jpcIdx, nextLine()) agar JPC melompat ke instruksi setelah blok THEN.

2.2.5 Generasi Loop

Implementasi generasi kode untuk struktur perulangan WHILE dan FOR.

```

1 void CodeGenerator::genWhileStmt(const ASTPtr& node) {
2     if (node->children.size() < 2)
3         throw CodeGenError("genWhileStmt: node WHILE_STMT butuh 2 anak.");
4
5     const ASTPtr& cond = node->children[0];
6     const ASTPtr& body = node->children[1];
7     int loopStart = nextLine();
8     genExpr(cond);
9     int jpcIdx = emit(OpCode::JPC, 0, 0);
10    genStatement(body);
11    emit(OpCode::JMP, 0, loopStart);
12    patch(jpcIdx, nextLine());
13 }
14
15 void CodeGenerator::genForStmt(const ASTPtr& node) {
16     if (node->children.size() < 4)
17         throw CodeGenError("genForStmt: node FOR_STMT butuh 4 anak.");
18
19     const ASTPtr& varRef = node->children[0];
20     const ASTPtr& startExpr = node->children[1];
21     const ASTPtr& endExpr = node->children[2];
22     const ASTPtr& body = node->children[3];
23     bool isDownto = (node->name == "downto");
24
25     if (varRef->kind != ASTNodeKind::VAR_REF || varRef->tabIndex < 0)
26         throw CodeGenError("genForStmt: variabel kontrol FOR tidak valid.");
27
28     int addr = varAddress(varRef->tabIndex);
29     int lev = levelDiff(varRef->tabIndex);
30
31     genExpr(startExpr);
32     emit(OpCode::STO, lev, addr);
33     int loopStart = nextLine();
34     emit(OpCode::LOD, lev, addr);
35     genExpr(endExpr);
36     emit(OpCode::OPR, 0, isDownto ? 10 : 12);
37     int jpcIdx = emit(OpCode::JPC, 0, 0);
38     genStatement(body);
39
40     emit(OpCode::LOD, lev, addr);
41     emit(OpCode::LIT, 0, 1);
42     emit(OpCode::OPR, 0, isDownto ? 3 : 2);
43     emit(OpCode::STO, lev, addr);
44
45     emit(OpCode::JMP, 0, loopStart);
46     patch(jpcIdx, nextLine());
47 }

```

Gambar 2.2.5 Kode genWhileStmt() dan genForStmt()

Berikut merupakan penjelasan dari kode `genWhileStmt()`:

- `loopStart = nextLine()`: merekam posisi awal kondisi loop.
- `genExpr(cond)` kemudian emit JPC. Jika kondisi false, lompat keluar loop.
- `genStatement(body)` untuk generate kode body loop.
- `emit (JMP 0, loopStart)` untuk lompat kembali ke evaluasi kondisi.
- `patch(jpcIdx, nextLine())` adalah isi alamat JPC ke setelah loop.

Berikut merupakan penjelasan dari kode `genForStmt()`:

- Inisialisasi dengan generate ekspresi start, STO ke variabel kontrol.
- Evaluasi kondisi LOD variabel kontrol, generate ekspresi end, OPR perbandingan (LEQ untuk TO, GEQ untuk DOWNT0).
- Increment/decrement setelah body, LOD variabel, LIT 1, OPR ADD/SUB, STO variabel kontrol.
- Mendukung arah TO (`isDownto = false`) dan DOWNT0 (`isDownto = true`).

2.2.6 Generasi Repeat-Until

Implementasi generasi kode untuk REPEAT-UNTIL dan CASE statement.

```

1 void CodeGenerator::genRepeatStmt(const ASTPtr& node) {
2     if (node->children.empty())
3         throw CodeGenError("genRepeatStmt: node REPEAT_STMT tidak memiliki anak.");
4     size_t condIdx = node->children.size() - 1;
5     int loopStart = nextLine();
6     for (size_t i = 0; i < condIdx; ++i) {
7         genStatement(node->children[i]);
8     }
9     genExpr(node->children[condIdx]);
10    emit(OpCode::JPC, 0, loopStart);
11 }
12
13 void CodeGenerator::genCaseStmt(const ASTPtr& node) {
14     if (node->children.size() < 2)
15         throw CodeGenError("genCaseStmt: CASE_STMT butuh selector dan blok.");
16
17     const ASTPtr& selector = node->children[0];
18
19     std::vector<int> jmpEndIdxs;
20
21     const ASTPtr* caseBlock = &node->children[1];
22     while (caseBlock && *caseBlock &&
23            (*caseBlock)->kind == ASTNodeKind::CASE_BLOCK) {
24         const ASTPtr& blk = *caseBlock;
25
26         genExpr(selector);
27         genExpr(blk->children[0]);
28         emit(OpCode::OPR, 0, 7);
29         int jpcIdx = emit(OpCode::JPC, 0, 0);
30
31         if (blk->children.size() >= 2)
32             genStatement(blk->children[1]);
33
34         jmpEndIdxs.push_back(emit(OpCode::JMP, 0, 0));
35         patch(jpcIdx, nextLine());
36
37         if (blk->children.size() >= 3)
38             caseBlock = &blk->children[2];
39         else
40             break;
41     }
42
43     int endAddr = nextLine();
44     for (int idx : jmpEndIdxs)
45         patch(idx, endAddr);
46 }

```

Gambar 2.2.6 Kode genRepeatStmt() dan genCaseStmt()

Berikut merupakan penjelasan dari kode genRepeatStmt():

- Anak terakhir dari node REPEAT_STMT adalah kondisi UNTIL; anak-anak sebelumnya adalah statement body.
- Mencatat loopStart = nextLine(), generate semua statement body, lalu generate kondisi.
- emit (JPC 0, loopStart), jika kondisi false, kembali ke awal loop. Berbeda dengan WHILE, REPEAT selalu mengeksekusi body minimal sekali.

Berikut merupakan penjelasan dari kode `genCaseStmt()`:

- Untuk setiap case block, generate ekspresi selector, generate nilai case, OPR EQ (perbandingan kesamaan), emit JPC jika tidak cocok.
- Jika cocok, maka generate statement blok tersebut, emit JMP ke akhir CASE (alamat di-patch nanti).
- Setelah semua case, patch semua JMP ke instruksi setelah CASE.

2.2.7 Pemanggilan Prosedur Built-in dan User-Defined

Fungsi ini menangani dua kategori pemanggilan prosedur.

```
1 void CodeGenerator::genProcCall(const ASTPtr& node) {
2     std::string name = node->name;
3     if (name == "writeln") {
4         if (node->children.empty()) {
5             int idx = (int)stringTable_.size();
6             stringTable_.push_back("");
7             emit(OpCode::LIT, 1, idx);
8             emit(OpCode::OPR, 0, 14);
9         } else {
10             for (size_t i = 0; i < node->children.size(); ++i) {
11                 genExpr(node->children[i]);
12                 if (i == node->children.size() - 1)
13                     emit(OpCode::OPR, 0, 14);
14                 else
15                     emit(OpCode::OPR, 0, 13);
16             }
17         }
18         return;
19     }
20
21     if (name == "write") {
22         for (const auto& arg : node->children) {
23             genExpr(arg);
24             emit(OpCode::OPR, 0, 13); // WRT tanpa newline
25         }
26         return;
27     }
28
29     if (name == "readln" || name == "read") {
30         return;
31     }
32
33     emit(OpCode::LIT, 0, 0);
34     emit(OpCode::LIT, 0, 0);
35     emit(OpCode::LIT, 0, 0);
36
37     for (const auto& arg : node->children) {
38         genExpr(arg);
39     }
40
41     int tabIdx = node->tabIndex;
42     if (tabIdx < 0)
43         throw CodeGenError("genProcCall: tabIndex prosedur '" + name + "' tidak valid.");
44
45     int procAddr = symtab_.tab()[tabIdx].adr;
46     int lev = levelDiff(tabIdx);
47     emit(OpCode::CAL, lev, procAddr);
48 }
```

Gambar 2.2.7 Kode `genProcCall()`

Berikut merupakan penjelasan dari kode genProcCall():

- Built-in writeln, untuk setiap argumen, generate ekspresi, lalu OPR 13 (WRT tanpa newline) untuk semua kecuali argumen terakhir, dan OPR 14 (WRITELN) untuk argumen terakhir. Writeln tanpa argumen mengirim string kosong dengan OPR 14.
- Built-in write, sama dengan writeln tetapi semua argumen menggunakan OPR 13.
- readln/read tidak men-generate instruksi (input diabaikan pada implementasi ini).
- emit tiga LIT 0, 0 sebagai slot SL, DL, RA pada frame baru, lalu generate semua argumen, kemudian emit CAL level, procAddr.

2.2.8 Generasi Kode untuk Nilai Literal

Menangani semua jenis konstanta literal dalam program Arion.

```
1 void CodeGenerator::genLiteral(const ASTPtr& node) {
2     switch (node->kind) {
3         case ASTNodeKind::CONST_INT: {
4             int val = 0;
5             try { val = std::stoi(node->name); }
6             catch (...) { throw CodeGenError("genLiteral: nilai integer tidak valid '" + node->name + "'"); }
7             emit(OpCode::LIT, 0, val);
8             break;
9         }
10
11         case ASTNodeKind::CONST_BOOL: {
12             std::string lower = node->name;
13             for (auto& c : lower) c = (char)tolower((unsigned char)c);
14             int val = (lower == "true") ? 1 : 0;
15             emit(OpCode::LIT, 0, val);
16             break;
17         }
18
19         case ASTNodeKind::CONST_CHAR: {
20             if (!node->name.empty()) {
21                 emit(OpCode::LIT, 0, (int)(unsigned char)node->name[0]);
22             } else {
23                 emit(OpCode::LIT, 0, 0);
24             }
25             break;
26         }
27
28         case ASTNodeKind::CONST_REAL: {
29             double val = 0.0;
30             try { val = std::stod(node->name); }
31             catch (...) {}
32             emit(OpCode::LIT, 0, static_cast<int>(val));
33             break;
34         }
35
36         case ASTNodeKind::CONST_STRING: {
37             int idx = (int)stringTable.size();
38             stringTable.push_back(node->name);
39             emit(OpCode::LIT, 1, idx);
40             break;
41         }
42
43         default:
44             throw CodeGenError("genLiteral: jenis literal tidak dikenal.");
45     }
46 }
```

Gambar 2.2.8 Kode genLiteral()

Berikut merupakan penjelasan dari kode genLiteral():

- CONST_INT: emit LIT 0, nilai_integer.
- CONST_BOOL: 'true' di-emit sebagai LIT 0, 1; 'false' sebagai LIT 0, 0.
- CONST_CHAR: dikonversi ke integer ASCII nya, emit LIT 0, ascii_value.
- CONST_REAL: dikonversi ke integer (truncate), emit LIT 0, nilai. Interpreter bekerja dengan integer.
- CONST_STRING: string ditambahkan ke stringTable_, dan emit LIT 1, indeks. Level=1 pada LIT menandakan ini adalah referensi string, bukan integer.

2.2.9 Deklarasi Subprogram

Men-generate kode untuk prosedur dan fungsi yang dideklarasikan pengguna.

```
1 void CodeGenerator::genProcDecl(const ASTPtr& node) {
2     int tabIdx = node->tabIndex;
3     if (tabIdx < 0)
4         throw CodeGenError("genProcDecl: tabIndex prosedur '" + node->name + "' tidak valid.");
5
6     int procStart = nextline();
7     const_cast<SymbolTable&>(syntab_).tab()[tabIdx].adr = procStart;
8     curlev_++;
9
10    int btabIdx = node->blockIndex;
11    if (btabIdx < 0) btabIdx = 1;
12
13    int fsize = frameSize(btabIdx);
14    emit(OpCode::INT, 0, fsize);
15
16    for (const auto& child : node->children) {
17        if (child->kind == ASTNodeKind::BLOCK ||
18            child->kind == ASTNodeKind::COMPOUND) {
19            genCompound(child);
20        }
21    }
22
23    emit(OpCode::RET, 0, 0);
24    curlev_--;
25 }
26
27 void CodeGenerator::genFuncDecl(const ASTPtr& node) {
28     int tabIdx = node->tabIndex;
29     if (tabIdx < 0)
30         throw CodeGenError("genFuncDecl: tabIndex fungsi '" + node->name + "' tidak valid.");
31
32     int funcStart = nextline();
33     const_cast<SymbolTable&>(syntab_).tab()[tabIdx].adr = funcStart;
34     curlev_++;
35
36     int btabIdx = node->blockIndex;
37     if (btabIdx < 0) btabIdx = 1;
38
39     int fsize = frameSize(btabIdx);
40     emit(OpCode::INT, 0, fsize);
41
42     for (const auto& child : node->children) {
43         if (child->kind == ASTNodeKind::BLOCK ||
44             child->kind == ASTNodeKind::COMPOUND) {
45             genCompound(child);
46         }
47     }
48
49     emit(OpCode::RET, 0, 1);
50     curlev_--;
51 }
52 }
```

Gambar 2.2.9 Kode genProcDecl() dan genFuncDecl()

Berikut merupakan penjelasan dari kode `genProcDecl()` dan `genFuncDecl()`:

- Merekam posisi instruksi pertama (`procStart/funcStart`) dan menyimpannya ke `syntab_tab()[tabIdx].adr`. Alamat ini nantinya digunakan oleh CAL saat pemanggilan.
- Menaikkan `curLev_++` untuk mengindikasikan kita masuk ke level leksikal yang lebih dalam.
- emit `INT 0, frameSize(btabIdx)` untuk mengalokasikan frame subprogram.
- Men-generate body subprogram melalui `genCompound()`.
- Prosedur menutup dengan emit `RET 0, 0`. Fungsi menutup dengan emit `RET 0, 1` (`operand=1` menandakan ada nilai balik).
- Menurunkan `curLev_--` setelah body subprogram selesai.

2.2.10 Mesin Eksekusi Utama

Loop eksekusi utama yang menjalankan setiap instruksi dari list intermediate code.

```

1 void Interpreter::execute() {
2     top_     = -1;
3     pc_      = 0;
4     base_    = 0;
5     callDepth_ = 0;
6     for (int i = 0; i < STACK_SIZE; ++i) stack_[i] = StackVal();
7
8     if (code_.empty()) return;
9
10    while (true) {
11        if (pc_ < 0 || pc_ >= (int)code_.size())
12            throw InvalidJumpError("PC di luar batas: " + std::to_string(pc_));
13
14        const Instruction& instr = code_[pc_];
15        ++pc_;
16
17        switch (instr.op) {
18
19            case OpCode::LIT:
20                if (instr.level == 1)
21                    pushStr(instr.operand);
22                else
23                    push(instr.operand);
24                break;
25
26            case OpCode::LOD: {
27                int b = ibase(instr.level);
28                int addr = b + instr.operand;
29                if (addr < 0 || addr >= STACK_SIZE)
30                    throw IndexOutOfBoundsError("LOD: alamat tidak valid " + std::to_string(addr));
31                if (top_ >= STACK_SIZE - 1)
32                    throw StackOverflowError("Stack penuh pada LOD PC=" + std::to_string(pc_-1));
33                stack_[++top_] = stack_[addr];
34                break;
35            }
36
37            case OpCode::STO: {
38                StackVal val = pop();
39                int b = ibase(instr.level);
40                int addr = b + instr.operand;
41                if (addr < 0 || addr >= STACK_SIZE)
42                    throw IndexOutOfBoundsError("STO: alamat tidak valid " + std::to_string(addr));
43                checkStackSmashing(addr, instr.level);
44                stack_[addr] = val;
45                break;
46            }
47
48            case OpCode::INT: {
49                int newTop = base_ + instr.operand - 1;
50                if (newTop >= STACK_SIZE)
51                    throw StackOverflowError("INT: alokasi frame melebihi batas stack");
52                top_ = newTop;
53                break;
54            }
55
56            case OpCode::JMP:
57                checkJumpTarget(instr.operand, "JMP");
58                pc_ = instr.operand;
59                break;
60
61            case OpCode::JPC: {
62                StackVal cond = pop();
63                if (cond.ival == 0) {
64                    checkJumpTarget(instr.operand, "JPC");
65                    pc_ = instr.operand;
66                }
67                break;
68            }
69        }
70    }
71 }

```

Gambar 2.2.10 (1) Kode execute()

```

1  case OpCode::CAL: {
2      if (callDepth_ >= MAX_CALL_DEPTH)
3          throw StackOverflowError("Kedalaman pemanggilan fungsi melebihi batas maksimum (" +
4                                  std::to_string(MAX_CALL_DEPTH) + " frame). Kemungkinan infinite recursion.");
5
6      int procAddr = instr.operand;
7      int psze = 0;
8      auto it = addrToPsze_.find(procAddr);
9      if (it != addrToPsze_.end()) psze = it->second;
10
11     int newBase = top_ - psze - 2;
12     if (newBase < 0)
13         throw StackUnderflowError("CAL: tidak cukup ruang untuk frame baru (newBase=" +
14                                 std::to_string(newBase) + ")");
15
16     stack_[newBase].ival = ibase(instr.level);
17     stack_[newBase].isStr = false;
18     stack_[newBase+1].ival = base_;
19     stack_[newBase+1].isStr = false;
20     stack_[newBase+2].ival = pc_;
21     stack_[newBase+2].isStr = false;
22
23     base_ = newBase;
24     pc_ = procAddr;
25     ++callDepth_;
26     break;
27 }
28
29 case OpCode::RET: {
30     if (base_ == 0) return;
31
32     int retPc = stack_[base_ + 2].ival;
33     int retBase = stack_[base_ + 1].ival;
34
35     if (retPc < 0 || retPc > (int)code_.size())
36         throw StackSmashingError("Return address " + std::to_string(retPc) +
37                                 " tidak valid - kemungkinan stack smashing terdeteksi");
38
39     StackVal retVal{};
40     bool hasRet = (instr.operand == 1);
41     if (hasRet) retVal = stack_[base_];
42
43     int expectedTop = base_ - 1;
44     top_ = expectedTop;
45     pc_ = retPc;
46     base_ = retBase;
47     --callDepth_;
48
49     if (hasRet) stack_[++top_] = retVal;
50     break;
51 }

```

Gambar 2.2.10 (2) Kode execute()

```
1      case OpCode::OPR: {
2          switch (instr.operand) {
3              case 0: break;
4
5              case 1: {
6                  StackVal a = pop();
7                  long long res = -(long long)a.ival;
8                  checkArithOverflow(res, "NEG");
9                  push((int)res);
10                 break;
11             }
12             case 2: {
13                 StackVal b = pop(); StackVal a = pop();
14                 long long res = (long long)a.ival + b.ival;
15                 checkArithOverflow(res, "ADD");
16                 push((int)res);
17                 break;
18             }
19             case 3: {
20                 StackVal b = pop(); StackVal a = pop();
21                 long long res = (long long)a.ival - b.ival;
22                 checkArithOverflow(res, "SUB");
23                 push((int)res);
24                 break;
25             }
26             case 4: {
27                 StackVal b = pop(); StackVal a = pop();
28                 long long res = (long long)a.ival * b.ival;
29                 checkArithOverflow(res, "MUL");
30                 push((int)res);
31                 break;
32             }
33             case 5: {
34                 StackVal b = pop(); StackVal a = pop();
35                 if (b.ival == 0)
36                     throw InterpreterError("Pembagian dengan nol pada PC=" + std::to_string(pc_1));
37                 push(a.ival / b.ival);
38                 break;
39             }
40             case 6: {
41                 StackVal b = pop(); StackVal a = pop();
42                 if (b.ival == 0)
43                     throw InterpreterError("Modulo dengan nol pada PC=" + std::to_string(pc_1));
44                 push(a.ival % b.ival);
45                 break;
46             }
47             case 7: { StackVal b=pop(); StackVal a=pop(); push(a.ival==b.ival?1:0); break; }
48             case 8: { StackVal b=pop(); StackVal a=pop(); push(a.ival!=b.ival?1:0); break; }
49             case 9: { StackVal b=pop(); StackVal a=pop(); push(a.ival<b.ival?1:0); break; }
50             case 10: { StackVal b=pop(); StackVal a=pop(); push(a.ival>b.ival?1:0); break; }
51             case 11: { StackVal b=pop(); StackVal a=pop(); push(a.ival>=b.ival?1:0); break; }
52             case 12: { StackVal b=pop(); StackVal a=pop(); push(a.ival<=b.ival?1:0); break; }
53
54             case 13: {
55                 StackVal val = pop();
56                 output_.push_back(valToString(val));
57                 break;
58             }
59             case 14: {
60                 StackVal val = pop();
61                 output_.push_back(valToString(val) + "\n");
62                 break;
63             }
64
65             default:
66                 throw InterpreterError("OPR: operand tidak dikenal " + std::to_string(instr.operand));
67         }
68         break;
69     }
70
71     default:
72         throw InterpreterError("Instruksi tidak dikenal pada PC=" + std::to_string(pc_1));
73 }
74 }
75 }
```

Gambar 2.2.10 (3) Kode execute()

Berikut merupakan penjelasan per instruksi dari kode execute():

- LIT: jika level==1 push string (pushStr), jika level==0 push integer (push).
- LOD: hitung base frame tujuan dengan ibase(level), kemudian copy nilai dari stack[base+operand] ke top of stack.
- STO: pop nilai dari top of stack, hitung base frame, simpan ke stack[base+operand]. Termasuk validasi checkStackSmashing untuk mencegah overwrite control words.
- INT: alokasikan frame dengan menggeser top_ ke base_ + operand - 1.
- JMP: set pc_ ke operand (setelah validasi checkJumpTarget).
- JPC: pop kondisi; jika ival==0 (false), jump ke operand.
- CAL: untuk activation record baru. Hitung newBase = top_ - psze - 2, isi slot SL (ibase(level)), DL (base_ saat ini), RA (pc_ saat ini). Set base_ = newBase, pc_ = procAddr, increment callDepth_.
- RET: ambil RA dan DL dari stack. Jika operand==1 (fungsi), simpan nilai balik (stack[base_]). Kembalikan top_, pc_, base_, decrement callDepth_.
- OPR: dispatcher untuk semua operasi. OPR 1=NEG, 2=ADD, 3=SUB, 4=MUL, 5=DIV, 6=MOD, 7=EQ, 8=NEQ, 9=LT, 10=GEQ, 11=GT, 12=LEQ, 13=WRT, 14=WRITELN.

2.2.11 Hierarki Exception dan Safety Checks



Gambar 2.2.11 (1) Kode interpreter.hpp



Gambar 2.2.11 (2) Kode `checkArithOverflow()`, `checkStackSmashing()`, `checkVariableAccess()`

Berikut merupakan penjelasan Hierarki Exception dan Safety Checks:

- InterpreterError, ketika base class semua error runtime interpreter, mewarisi std::runtime_error.
- StackOverflowError, ketika stack penuh ($\text{top_} \geq \text{STACK_SIZE} - 1$) atau kedalaman call melebihi MAX_CALL_DEPTH (1000).
- StackUnderflowError, saat pop dari stack kosong ($\text{top_} < 0$).
- StackSmashingError, saat penulisan ke slot SL/DL/RA (control words) frame aktif secara tidak sah.
- IndexOutOfBoundsError: akses di luar batas stack atau frame.
- InvalidJumpError, ketika PC atau jump target di luar batas array instruksi.
- OverflowError / UnderflowError, ketika hasil operasi aritmetika melampaui batas int32.
- checkArithOverflow() untuk mendeteksi overflow sebelum cast ke int dengan menggunakan long long.

BAB III

PENGUJIAN

3.1 Test Case 1

Masukan (input)
<pre>program TestKeywords; var x : integer; y : integer; begin x := 10; if x > 5 then y := x * 2 else y := x div 2; writeln(y); while y > 0 do begin y := y - 1; writeln(y); end; end.</pre>
Keluaran (output)
<pre>INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 5 1 LIT 0 10 2 STO 0 3 3 LOD 0 3 4 LIT 0 5 5 OPR 0 11 6 JPC 0 12 7 LOD 0 3</pre>

8	LIT	0	2
9	OPR	0	4
10	STO	0	4
11	JMP	0	16
12	LOD	0	3
13	LIT	0	2
14	OPR	0	5
15	STO	0	4
16	LOD	0	4
17	OPR	0	14
18	LOD	0	4
19	LIT	0	0
20	OPR	0	11
21	JPC	0	29
22	LOD	0	4
23	LIT	0	1
24	OPR	0	3
25	STO	0	4
26	LOD	0	4
27	OPR	0	14
28	JMP	0	18
29	RET	0	0

20

19

18

17

16

15

14

13

12

11

10

9

8

7

6
5
4
3
2
1
0

Tabel 3.1 Input/Output Test Case 1

3.2 Test Case 2

Masukan (input)
<pre> program TestArith; var a : integer; b : integer; c : integer; begin a := 10; b := 3; c := a + b; writeln(c); c := a - b; writeln(c); c := a * b; writeln(c); c := a div b; writeln(c); c := a mod b; writeln(c) end. </pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 6 </pre>

1	LIT	0	10
2	STO	0	3
3	LIT	0	3
4	STO	0	4
5	LOD	0	3
6	LOD	0	4
7	OPR	0	2
8	STO	0	5
9	LOD	0	5
10	OPR	0	14
11	LOD	0	3
12	LOD	0	4
13	OPR	0	3
14	STO	0	5
15	LOD	0	5
16	OPR	0	14
17	LOD	0	3
18	LOD	0	4
19	OPR	0	4
20	STO	0	5
21	LOD	0	5
22	OPR	0	14
23	LOD	0	3
24	LOD	0	4
25	OPR	0	5
26	STO	0	5
27	LOD	0	5
28	OPR	0	14
29	LOD	0	3
30	LOD	0	4
31	OPR	0	6
32	STO	0	5
33	LOD	0	5
34	OPR	0	14
35	RET	0	0

7
30
3
1

Tabel 3.2 Input/Output Test Case 2

3.3 Test Case 3

Masukan (input)
<pre>program TestLogic; var x : integer; y : integer; flag : boolean; begin x := 5; y := 10; flag := (x < y) and (y > 0); if flag then writeln(1) else writeln(0); flag := not flag; if flag then writeln(1) else writeln(0); flag := (x > y) or (x = 5); if flag then writeln(1) else writeln(0) end.</pre>
Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	6
1	LIT	0	5
2	STO	0	3
3	LIT	0	10
4	STO	0	4
5	LOD	0	3
6	LOD	0	4
7	OPR	0	9
8	LOD	0	4
9	LIT	0	0
10	OPR	0	11
11	OPR	0	4
12	STO	0	5
13	LOD	0	5
14	JPC	0	18
15	LIT	0	1
16	OPR	0	14
17	JMP	0	20
18	LIT	0	0
19	OPR	0	14
20	LOD	0	5
21	OPR	0	1
22	LIT	0	1
23	OPR	0	2
24	STO	0	5
25	LOD	0	5
26	JPC	0	30
27	LIT	0	1
28	OPR	0	14
29	JMP	0	32
30	LIT	0	0
31	OPR	0	14
32	LOD	0	3
33	LOD	0	4
34	OPR	0	11
35	LOD	0	3
36	LIT	0	5

37	OPR	0	7
38	OPR	0	2
39	LIT	0	0
40	OPR	0	11
41	STO	0	5
42	LOD	0	5
43	JPC	0	47
44	LIT	0	1
45	OPR	0	14
46	JMP	0	49
47	LIT	0	0
48	OPR	0	14
49	RET	0	0
1			
0			
1			

Tabel 3.3 Input/Output Test Case 3

3.4 Test Case 4

Masukan (input)
<pre> program TestIfElse; var n : integer; result : integer; begin n := 7; if n > 10 then result := 1 else if n > 5 then result := 2 else result := 3; writeln(result); </pre>


```
n := 15;
if n > 10 then
    result := 1
else if n > 5 then
    result := 2
else
    result := 3;
writeln(result)
end.
```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	5
1	LIT	0	7
2	STO	0	3
3	LOD	0	3
4	LIT	0	10
5	OPR	0	11
6	JPC	0	10
7	LIT	0	1
8	STO	0	4
9	JMP	0	19
10	LOD	0	3
11	LIT	0	5
12	OPR	0	11
13	JPC	0	17
14	LIT	0	2
15	STO	0	4
16	JMP	0	19
17	LIT	0	3
18	STO	0	4
19	LOD	0	4
20	OPR	0	14
21	LIT	0	15
22	STO	0	3
23	LOD	0	3

24	LIT	0	10
25	OPR	0	11
26	JPC	0	30
27	LIT	0	1
28	STO	0	4
29	JMP	0	39
30	LOD	0	3
31	LIT	0	5
32	OPR	0	11
33	JPC	0	37
34	LIT	0	2
35	STO	0	4
36	JMP	0	39
37	LIT	0	3
38	STO	0	4
39	LOD	0	4
40	OPR	0	14
41	RET	0	0

2

1

Tabel 3.4 Input/Output Test Case 4

3.5 Test Case 5

Masukan (input)
<pre> program TestWhile; var i : integer; sum : integer; begin i := 1; sum := 0; repeat sum := sum + i; </pre>

```

    i := i + 1;
until i > 10;
writeln(sum);
end.

```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	5
1	LIT	0	1
2	STO	0	3
3	LIT	0	0
4	STO	0	4
5	LOD	0	4
6	LOD	0	3
7	OPR	0	2
8	STO	0	4
9	LOD	0	3
10	LIT	0	1
11	OPR	0	2
12	STO	0	3
13	LOD	0	3
14	LIT	0	10
15	OPR	0	11
16	JPC	0	5
17	LOD	0	4
18	OPR	0	14
19	RET	0	0

55

Tabel 3.5 Input/Output Test Case 5

3.6 Test Case 6

Masukan (input)

```

program TestRepeat;
var
  n : integer;
  product : integer;
begin
  n := 1;
  product := 1;
  repeat
    product := product * n;
    n := n + 1
  until n > 5;
  writeln(product)
end.

```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	5
1	LIT	0	1
2	STO	0	3
3	LIT	0	1
4	STO	0	4
5	LOD	0	4
6	LOD	0	3
7	OPR	0	4
8	STO	0	4
9	LOD	0	3
10	LIT	0	1
11	OPR	0	2
12	STO	0	3
13	LOD	0	3
14	LIT	0	5
15	OPR	0	11
16	JPC	0	5
17	LOD	0	4
18	OPR	0	14
19	RET	0	0

Tabel 3.6 Input/Output Test Case 6

3.7 Test Case 7

Masukan (input)																																											
<pre> program TestProc; var total : integer; procedure addTo(val: integer); begin total := total + val end; begin total := 0; addTo(5); addTo(10); addTo(3); writeln(total) end. </pre>																																											
Keluaran (output)																																											
INTERMEDIATE CODE (STACK MACHINE) <table> <tr><td>0</td><td>INT</td><td>0</td><td>4</td></tr> <tr><td>1</td><td>JMP</td><td>0</td><td>8</td></tr> <tr><td>2</td><td>INT</td><td>0</td><td>4</td></tr> <tr><td>3</td><td>LOD</td><td>1</td><td>3</td></tr> <tr><td>4</td><td>LOD</td><td>0</td><td>3</td></tr> <tr><td>5</td><td>OPR</td><td>0</td><td>2</td></tr> <tr><td>6</td><td>STO</td><td>1</td><td>3</td></tr> <tr><td>7</td><td>RET</td><td>0</td><td>0</td></tr> <tr><td>8</td><td>LIT</td><td>0</td><td>0</td></tr> <tr><td>9</td><td>STO</td><td>0</td><td>3</td></tr> </table>				0	INT	0	4	1	JMP	0	8	2	INT	0	4	3	LOD	1	3	4	LOD	0	3	5	OPR	0	2	6	STO	1	3	7	RET	0	0	8	LIT	0	0	9	STO	0	3
0	INT	0	4																																								
1	JMP	0	8																																								
2	INT	0	4																																								
3	LOD	1	3																																								
4	LOD	0	3																																								
5	OPR	0	2																																								
6	STO	1	3																																								
7	RET	0	0																																								
8	LIT	0	0																																								
9	STO	0	3																																								

10	LIT	0	0
11	LIT	0	0
12	LIT	0	0
13	LIT	0	5
14	CAL	0	2
15	LIT	0	0
16	LIT	0	0
17	LIT	0	0
18	LIT	0	10
19	CAL	0	2
20	LIT	0	0
21	LIT	0	0
22	LIT	0	0
23	LIT	0	3
24	CAL	0	2
25	LOD	0	3
26	OPR	0	14
27	RET	0	0

18

Tabel 3.7 Input/Output Test Case 7

3.8 Test Case 8

Masukan (input)
<pre> program TestFunc; var r : integer; function square(n: integer): integer; begin square := n * n end; function add(a: integer; b: integer): integer; begin add := a + b </pre>

```
end;  
begin  
  r := square(6);  
  writeln(r);  
  r := add(square(3), square(4));  
  writeln(r)  
end.
```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	4
1	JMP	0	14
2	INT	0	4
3	LOD	0	3
4	LOD	0	3
5	OPR	0	4
6	STO	0	0
7	RET	0	1
8	INT	0	5
9	LOD	0	3
10	LOD	0	4
11	OPR	0	2
12	STO	0	0
13	RET	0	1
14	LIT	0	0
15	LIT	0	0
16	LIT	0	0
17	LIT	0	6
18	CAL	0	2
19	STO	0	3
20	LOD	0	3
21	OPR	0	14
22	LIT	0	0
23	LIT	0	0
24	LIT	0	0
25	LIT	0	0

26	LIT	0	0
27	LIT	0	0
28	LIT	0	3
29	CAL	0	2
30	LIT	0	0
31	LIT	0	0
32	LIT	0	0
33	LIT	0	4
34	CAL	0	2
35	CAL	0	8
36	STO	0	3
37	LOD	0	3
38	OPR	0	14
39	RET	0	0

36

25

Tabel 3.8 Input/Output Test Case 8

3.9 Test Case 9

Masukan (input)
<pre> program TestCase; var x : integer; result : integer; begin x := 3; result := 0; case x of 1: result := 100; 2: result := 200; 3: result := 300; 4: result := 400 end; </pre>


```
writeln(result);  
x := 99;  
result := 0;  
case x of  
  1: result := 100;  
  2: result := 200  
end;  
writeln(result)  
end.
```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	5
1	LIT	0	3
2	STO	0	3
3	LIT	0	0
4	STO	0	4
5	LOD	0	3
6	LIT	0	1
7	OPR	0	7
8	JPC	0	12
9	LIT	0	100
10	STO	0	4
11	JMP	0	33
12	LOD	0	3
13	LIT	0	2
14	OPR	0	7
15	JPC	0	19
16	LIT	0	200
17	STO	0	4
18	JMP	0	33
19	LOD	0	3
20	LIT	0	3
21	OPR	0	7
22	JPC	0	26
23	LIT	0	300

24	STO	0	4
25	JMP	0	33
26	LOD	0	3
27	LIT	0	4
28	OPR	0	7
29	JPC	0	33
30	LIT	0	400
31	STO	0	4
32	JMP	0	33
33	LOD	0	4
34	OPR	0	14
35	LIT	0	99
36	STO	0	3
37	LIT	0	0
38	STO	0	4
39	LOD	0	3
40	LIT	0	1
41	OPR	0	7
42	JPC	0	46
43	LIT	0	100
44	STO	0	4
45	JMP	0	53
46	LOD	0	3
47	LIT	0	2
48	OPR	0	7
49	JPC	0	53
50	LIT	0	200
51	STO	0	4
52	JMP	0	53
53	LOD	0	4
54	OPR	0	14
55	RET	0	0

300

0

Tabel 3.9 Input/Output Test Case 9

3.10 Test Case 10

Masukan (input)
<pre> program TestConst; const MAX == 100; MIN == 1; var x : integer; y : integer; begin x := MAX; y := MIN; writeln(x); writeln(y); writeln(x + y); writeln(x - y); writeln(x div 10) end.</pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 5 1 LIT 0 100 2 STO 0 3 3 LIT 0 1 4 STO 0 4 5 LOD 0 3 6 OPR 0 14 7 LOD 0 4 8 OPR 0 14 9 LOD 0 3 10 LOD 0 4 11 OPR 0 2</pre>

12	OPR	0	14
13	LOD	0	3
14	LOD	0	4
15	OPR	0	3
16	OPR	0	14
17	LOD	0	3
18	LIT	0	10
19	OPR	0	5
20	OPR	0	14
21	RET	0	0
100			
1			
101			
99			
10			

Tabel 3.10 Input/Output Test Case 10

3.11 Test Case 11

Masukan (input)
<pre> program TestMultiProc; var total : integer; procedure addVal(n: integer); begin total := total + n; end; procedure subVal(n: integer); begin total := total - n; end; begin total := 100; addVal(20); </pre>

```
addVal(30);  
subVal(10);  
writeln(total);  
end.
```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	4
1	JMP	0	14
2	INT	0	4
3	LOD	1	3
4	LOD	0	3
5	OPR	0	2
6	STO	1	3
7	RET	0	0
8	INT	0	4
9	LOD	1	3
10	LOD	0	3
11	OPR	0	3
12	STO	1	3
13	RET	0	0
14	LIT	0	100
15	STO	0	3
16	LIT	0	0
17	LIT	0	0
18	LIT	0	0
19	LIT	0	20
20	CAL	0	2
21	LIT	0	0
22	LIT	0	0
23	LIT	0	0
24	LIT	0	30
25	CAL	0	2
26	LIT	0	0
27	LIT	0	0
28	LIT	0	0

29	LIT	0	10
30	CAL	0	8
31	LOD	0	3
32	OPR	0	14
33	RET	0	0
140			

Tabel 3.11 Input/Output Test Case 11

3.12 Test Case 12

Masukan (input)
<pre> program TestWriteString; var x : integer; y : integer; begin x := 42; y := 100; writeln('Hello World'); writeln('x = ', x); writeln('y = ', y); writeln('Sum = ', x + y); writeln('Done'); end.</pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 5 1 LIT 0 42 2 STO 0 3 3 LIT 0 100 4 STO 0 4 5 LIT 1 0</pre>

6	OPR	0	14
7	LIT	1	1
8	OPR	0	13
9	LOD	0	3
10	OPR	0	14
11	LIT	1	2
12	OPR	0	13
13	LOD	0	4
14	OPR	0	14
15	LIT	1	3
16	OPR	0	13
17	LOD	0	3
18	LOD	0	4
19	OPR	0	2
20	OPR	0	14
21	LIT	1	4
22	OPR	0	14
23	RET	0	0
Hello World			
x = 42			
y = 100			
Sum = 142			
Done			

Tabel 3.12 Input/Output Test Case 12

3.13 Test Case 13

Masukan (input)
<pre> program TestRecord; type Point == record x: integer; y: integer end; </pre>

```

var
  p : Point;
  q : Point;
begin
  p.x := 3;
  p.y := 4;
  q.x := p.x + 1;
  q.y := p.y + 1;
  writeln(p.x);
  writeln(p.y);
  writeln(q.x);
  writeln(q.y)
end.

```

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	7
1	LIT	0	3
2	STO	0	3
3	LIT	0	4
4	STO	0	4
5	LOD	0	3
6	LIT	0	1
7	OPR	0	2
8	STO	0	5
9	LOD	0	4
10	LIT	0	1
11	OPR	0	2
12	STO	0	6
13	LOD	0	3
14	OPR	0	14
15	LOD	0	4
16	OPR	0	14
17	LOD	0	5
18	OPR	0	14
19	LOD	0	6

20	OPR	0	14
21	RET	0	0
3			
4			
4			
5			

Tabel 3.13 Input/Output Test Case 13

3.14 Test Case 14

Masukan (input)
<pre> program TestFor; var i : integer; s : integer; begin s := 0; for i := 1 to 10 do begin s := s + i; end; writeln(s); for i := 5 downto 1 do writeln(i); end. </pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 5 1 LIT 0 0 2 STO 0 4 3 LIT 0 1 4 STO 0 3 </pre>

5	LOD	0	3
6	LIT	0	10
7	OPR	0	12
8	JPC	0	18
9	LOD	0	4
10	LOD	0	3
11	OPR	0	2
12	STO	0	4
13	LOD	0	3
14	LIT	0	1
15	OPR	0	2
16	STO	0	3
17	JMP	0	5
18	LOD	0	4
19	OPR	0	14
20	LIT	0	5
21	STO	0	3
22	LOD	0	3
23	LIT	0	1
24	OPR	0	10
25	JPC	0	33
26	LOD	0	3
27	OPR	0	14
28	LOD	0	3
29	LIT	0	1
30	OPR	0	3
31	STO	0	3
32	JMP	0	22
33	RET	0	0

55

5

4

3

2

1

Tabel 3.14 Input/Output Test Case 14

3.15 Test Case 15

Masukan (input)
<pre> program CoverAll; var a : integer; b : integer; result : integer; i : integer; flag : boolean; check : integer; begin a := 10; b := 3; result := a * b; result := result + a; result := result - b; result := result div b; result := result mod b; writeln(result); flag := (a > b) and (result >= 0); if flag then check := a - b else check := b - a; if not flag then check := 0; writeln(check); i := 1; repeat writeln(i); i := i + 1 until i > 3; writeln(i) end. </pre>

Keluaran (output)

INTERMEDIATE CODE (STACK MACHINE)

0	INT	0	9
1	LIT	0	10
2	STO	0	3
3	LIT	0	3
4	STO	0	4
5	LOD	0	3
6	LOD	0	4
7	OPR	0	4
8	STO	0	5
9	LOD	0	5
10	LOD	0	3
11	OPR	0	2
12	STO	0	5
13	LOD	0	5
14	LOD	0	4
15	OPR	0	3
16	STO	0	5
17	LOD	0	5
18	LOD	0	4
19	OPR	0	5
20	STO	0	5
21	LOD	0	5
22	LOD	0	4
23	OPR	0	6
24	STO	0	5
25	LOD	0	5
26	OPR	0	14
27	LOD	0	3
28	LOD	0	4
29	OPR	0	11
30	LOD	0	5
31	LIT	0	0

32	OPR	0	10
33	OPR	0	4
34	STO	0	7
35	LOD	0	7
36	JPC	0	42
37	LOD	0	3
38	LOD	0	4
39	OPR	0	3
40	STO	0	8
41	JMP	0	46
42	LOD	0	4
43	LOD	0	3
44	OPR	0	3
45	STO	0	8
46	LOD	0	7
47	OPR	0	1
48	LIT	0	1
49	OPR	0	2
50	JPC	0	53
51	LIT	0	0
52	STO	0	8
53	LOD	0	8
54	OPR	0	14
55	LIT	0	1
56	STO	0	6
57	LOD	0	6
58	OPR	0	14
59	LOD	0	6
60	LIT	0	1
61	OPR	0	2
62	STO	0	6
63	LOD	0	6
64	LIT	0	3
65	OPR	0	11
66	JPC	0	57
67	LOD	0	6
68	OPR	0	14
69	RET	0	0

0
7
1
2
3
4

Tabel 3.13 Input/Output Test Case 15

3.16 Test Case 16

Masukan (input)																																							
<pre> program TestDivByZero; var a : integer; b : integer; begin a := 10; b := 0; a := a div b; writeln(a); end.</pre>																																							
Keluaran (output)																																							
INTERMEDIATE CODE (STACK MACHINE) <table> <tr><td>0</td><td>INT</td><td>0</td><td>5</td></tr> <tr><td>1</td><td>LIT</td><td>0</td><td>10</td></tr> <tr><td>2</td><td>STO</td><td>0</td><td>3</td></tr> <tr><td>3</td><td>LIT</td><td>0</td><td>0</td></tr> <tr><td>4</td><td>STO</td><td>0</td><td>4</td></tr> <tr><td>5</td><td>LOD</td><td>0</td><td>3</td></tr> <tr><td>6</td><td>LOD</td><td>0</td><td>4</td></tr> <tr><td>7</td><td>OPR</td><td>0</td><td>5</td></tr> <tr><td>8</td><td>STO</td><td>0</td><td>3</td></tr> </table>				0	INT	0	5	1	LIT	0	10	2	STO	0	3	3	LIT	0	0	4	STO	0	4	5	LOD	0	3	6	LOD	0	4	7	OPR	0	5	8	STO	0	3
0	INT	0	5																																				
1	LIT	0	10																																				
2	STO	0	3																																				
3	LIT	0	0																																				
4	STO	0	4																																				
5	LOD	0	3																																				
6	LOD	0	4																																				
7	OPR	0	5																																				
8	STO	0	3																																				

9	LOD	0	3
10	OPR	0	14
11	RET	0	0
[INTERPRETER ERROR] Pembagian dengan nol pada PC=7			

Tabel 3.16 Input/Output Test Case 16

3.17 Test Case 17

Masukan (input)			
<pre> program TestModByZero; var a : integer; b : integer; begin a := 10; b := 0; a := a mod b; writeln(a); end. </pre>			
Keluaran (output)			
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 5 1 LIT 0 10 2 STO 0 3 3 LIT 0 0 4 STO 0 4 5 LOD 0 3 6 LOD 0 4 7 OPR 0 6 8 STO 0 3 9 LOD 0 3 10 OPR 0 14 11 RET 0 0 </pre>			

[INTERPRETER ERROR] Modulo dengan nol pada PC=7

Tabel 3.17 Input/Output Test Case 17

3.18 Test Case 18

Masukan (input)			
<pre>program TestStackOverflow; var n : integer; function inf(x : integer) : integer; begin inf := inf(x + 1); end; begin n := inf(0); writeln(n); end.</pre>			
Keluaran (output)			
INTERMEDIATE CODE (STACK MACHINE)			
0	INT	0	4
1	JMP	0	12
2	INT	0	4
3	LIT	0	0
4	LIT	0	0
5	LIT	0	0
6	LOD	0	3
7	LIT	0	1
8	OPR	0	2
9	CAL	1	2
10	STO	0	0
11	RET	0	1
12	LIT	0	0
13	LIT	0	0

14	LIT	0	0
15	LIT	0	0
16	CAL	0	2
17	STO	0	3
18	LOD	0	3
19	OPR	0	14
20	RET	0	0

[INTERPRETER ERROR] StackOverflowError: Kedalaman pemanggilan fungsi melebihi batas maksimum (1000 frame). Kemungkinan infinite recursion.

Tabel 3.18 Input/Output Test Case 18

3.19 Test Case 19

Masukan (input)
<pre> program TestIntOverflow; var a : integer; begin a := 2147483647; a := a + 1; writeln(a); end. </pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 4 1 LIT 0 2147483647 2 STO 0 3 3 LOD 0 3 4 LIT 0 1 5 OPR 0 2 6 STO 0 3 7 LOD 0 3 </pre>

8	OPR	0	14
9	RET	0	0
[INTERPRETER ERROR] OverflowError: Operasi ADD menghasilkan nilai 2147483648 melebihi batas maksimum integer (2147483647)			

Tabel 3.19 Input/Output Test Case 19

3.20 Test Case 20

Masukan (input)
<pre> program TestIntUnderflow; var a : integer; begin a := -2147483648; a := a - 1; writeln(a); end. </pre>
Keluaran (output)
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 4 1 LIT 0 -2147483648 2 STO 0 3 3 LOD 0 3 4 LIT 0 1 5 OPR 0 3 6 STO 0 3 7 LOD 0 3 8 OPR 0 14 9 RET 0 0 [INTERPRETER ERROR] UnderflowError: Operasi SUB menghasilkan nilai -2147483649 di bawah batas minimum integer (-2147483648) </pre>

Tabel 3.20 Input/Output Test Case 20

3.21 Test Case 21

Masukan (input)	
<pre> program TestMulOverflow; var a : integer; begin a := 2147483647; a := a * 2; writeln(a); end. </pre>	
Keluaran (output)	
<pre> INTERMEDIATE CODE (STACK MACHINE) 0 INT 0 4 1 LIT 0 2147483647 2 STO 0 3 3 LOD 0 3 4 LIT 0 2 5 OPR 0 4 6 STO 0 3 7 LOD 0 3 8 OPR 0 14 9 RET 0 0 [INTERPRETER ERROR] OverflowError: Operasi MUL menghasilkan nilai 4294967294 melebihi batas maksimum integer (2147483647) </pre>	

Tabel 3.21 Input/Output Test Case 21

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 4 ini, implementasi Arion Interpreter telah berhasil diselesaikan dengan dua komponen back-end utama yang berfungsi penuh, yaitu Intermediate Code Generator dan Interpreter (Executor). Beberapa kesimpulan yang dapat diambil dari pengerjaan milestone ini adalah:

1. Intermediate Code Generator berhasil diimplementasikan menggunakan model PL/0-inspired stack machine dengan 9 OpCode (LIT, LOD, STO, INT, JMP, JPC, OPR, CAL, RET). Generator mampu menangani seluruh konstruksi bahasa Arion yang telah didefinisikan, termasuk assignment, ekspresi aritmetika/logika, percabangan (IF-ELSE, CASE), perulangan (FOR, WHILE, REPEAT-UNTIL), serta deklarasi dan pemanggilan prosedur/fungsi.
2. Teknik backpatching telah berhasil diterapkan untuk menangani forward jump pada instruksi JPC dan JMP di struktur IF-ELSE, WHILE, FOR, REPEAT, dan CASE. Kemudian, fungsi patch() memungkinkan pengisian alamat tujuan secara retroaktif setelah blok kode terkait selesai di-generate.
3. Model activation record yang digunakan (3 control words: SL, DL, RA + parameter + variabel lokal) terbukti mendukung rekursi dan *nested scope* dengan benar. Mekanisme static chain (field SL) memungkinkan akses variabel dari scope luar melalui instruksi LOD/STO dengan level difference > 0 .
4. Interpreter mengimplementasikan mesin virtual stack yang *robust* dengan kapasitas 32.768 slot dan dukungan *call depth* hingga 1.000 frame. Dengan hierarki exception yang komprehensif, memastikan program gagal secara *graceful* dengan pesan error yang informatif.
5. Arsitektur front-end/back-end yang terpisah terbukti efektif. Back-end sepenuhnya bergantung pada Decorated AST dan Symbol Table tanpa mengetahui sintaksis bahasa Arion, yang sesuai dengan prinsip desain compiler modern.

4.2 Saran

Berdasarkan dari pengalaman mengerjakan Milestone 4, terdapat beberapa saran yang dapat menjadi pertimbangan untuk pengembangan, antara lain:

1. Melakukan optimasi pada code generator. Teknik seperti constant folding (melipat ekspresi konstanta saat compile-time), dead code elimination, dan peephole optimization dapat diterapkan untuk menghasilkan intermediate code yang lebih efisien.
2. Memastikan dukungan tipe data yang lebih lengkap. Operasi real/float saat ini di-truncate ke integer. Pengembangan dapat menambahkan dukungan floating-point murni pada StackVal (menambahkan field double atau menggunakan union) dan menambahkan OpCode baru untuk operasi float.
3. Membuat dukungan array dan record yang lebih lengkap. Implementasi saat ini memperlakukan array dan record access secara terbatas, hanya menggunakan alamat basis saja tanpa offset dinamis. Pengembangan penuh membutuhkan instruksi ALOAD/ASTORE dengan kalkulasi offset runtime.
4. Membuat input interaktif (readln/read). Instruksi read/readln saat ini tidak menghasilkan kode apapun. Penambahan OpCode INP untuk membaca nilai dari stdin akan membuat interpreter lebih fungsional untuk program interaktif.
5. Penambahan suite unit test otomatis (misalnya menggunakan Google Test) untuk setiap fungsi genXxx() dan setiap OpCode interpreter akan meningkatkan reliabilitas dan kemudahan deteksi regresi di masa depan.

LAMPIRAN

Link Repository : <https://github.com/gabriellaalubis/OTH-Tubes-IF2224-2026>

Pembagian tugas :

Nama	NIM	Pembagian	Persentase
Gabriella Botimada Lubis	13524006	● Implementasi CodeGenerator dan Interpreter ● Pengujian Test Case ● Penyusunan Laporan ● Tester	33.3%
Stefani Angeline Oroh	13524064	● Implementasi CodeGenerator dan Interpreter ● Pengujian Test Case ● Penyusunan Laporan ● Tester	33.3%
Reva Natania Sitohang	13524098	● Implementasi CodeGenerator dan Interpreter ● Pengujian Test Case ● Penyusunan Laporan ● Tester	33.3%

REFERENSI

- [1] GeeksforGeeks, "Intermediate Code Generation in Compiler Design." [Online]. Available: <https://www.geeksforgeeks.org/intermediate-code-generation-in-compiler-design/>. [Accessed: Jun. 2026].
- [2] GeeksforGeeks, "Symbol Table in Compiler." [Online]. Available: <https://www.geeksforgeeks.org/symbol-table-compiler/>. [Accessed: Jun. 2026].
- [3] TutorialsPoint, "Compiler Design Code Optimization." [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_code_optimization.htm. [Accessed: Jun. 2026].
- [4] cppreference.com, "std::runtime_error C++ Standard Library." [Online]. Available: https://en.cppreference.com/w/cpp/error/runtime_error. [Accessed: Jun. 2026].